

Physlib Monthly Report

1–31 May 2024

Contributors: Joseph Tooby-Smith, Pietro Monticone

Reviewers: Joseph Tooby-Smith

Generated 14 August 2026 from commit [801a929](#).

Abstract

Physlib is a community library of formalised physics for the [Lean 4](#) proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month’s additions to it.

It covers 1–31 May 2024 on branch `master` of [leanprover-community/physlib](#), between commits [22d784b](#) and [801a929](#). Over that period 2 contributors landed 134 commits, changing 3,117 lines across 54 files and adding 266 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at [GitHub compare](#).

Contents

1	Introduction	2
1.1	Contributors	2
1.2	Reviewers	2
1.3	Modified subfolders	2
1.4	New declarations by kind	3
2	Details by file	3
2.1	<code>HepLean.FlavorPhysics.CKMMatrix.Basic</code>	3
2.2	<code>HepLean.GroupTheory.S03.Basic</code>	3
2.3	<code>HepLean.SpaceTime.Basic</code>	5
2.4	<code>HepLean.SpaceTime.CliffordAlgebra</code>	7
2.5	<code>HepLean.SpaceTime.FourVelocity</code>	8
2.6	<code>HepLean.SpaceTime.LorentzAlgebra.Basic</code>	11
2.7	<code>HepLean.SpaceTime.LorentzGroup.Basic</code>	12
2.8	<code>HepLean.SpaceTime.LorentzGroup.Boosts</code>	14
2.9	<code>HepLean.SpaceTime.LorentzGroup.Orthochronous</code>	16
2.10	<code>HepLean.SpaceTime.LorentzGroup.Proper</code>	18
2.11	<code>HepLean.SpaceTime.LorentzGroup.Rotations</code>	20
2.12	<code>HepLean.SpaceTime.Metric</code>	20
2.13	<code>HepLean.StandardModel.Basic</code>	23
2.14	<code>HepLean.StandardModel.HiggsBoson.Basic</code>	23
2.15	<code>HepLean.StandardModel.HiggsBoson.TargetSpace</code>	26
2.16	<code>HepLean.StandardModel.Representations</code>	31

1 Introduction

During May 2024, 2 contributors submitted 134 commits that changed 3,117 lines (**+3,022 / -95**) across 54 files spanning 17 top-level areas of the repository. Of those, **22** files were newly added. This report catalogues 266 newly added Lean declarations: 171 lemmas, 69 defs, 21 instances, 4 abbrevs, and 1 theorem. We're delighted to recognize the following first-time contributor this month: [Pietro Monticone](#).

See the [full compare diff](#) and the [list of commits](#) on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
Joseph Tooby-Smith	7,329	107
Pietro Monticone	120	27

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

Reviewer	PRs Reviewed
Joseph Tooby-Smith	1

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
HepLean/SpaceTime	10	+10	—	+1,588	-0
HepLean/StandardModel	4	+4	—	+694	-0
HepLean/GroupTheory	1	+1	—	+234	-0
LICENSE	1	+1	—	+201	-0
HepLean/AnomalyCancellation	19	—	—	+58	-55
scripts	5	+4	—	+112	-0
README.md	1	—	—	+53	-12
lake-manifest.json	1	—	—	+15	-15
CONTRIBUTING.md	1	+1	—	+23	-0
HepLean.lean	1	—	—	+15	-0
HepLean/FlavorPhysics	2	—	—	+7	-7
lakefile.toml	1	+1	—	+14	-0
.github/workflows	2	—	—	+5	-2
.gitignore	1	—	—	+1	-2
.lake	2	—	—	+1	-1
lean-toolchain	1	—	—	+1	-1
lakefile.lean.bak	1	—	—	+0	-0

1.4 New declarations by kind

Kind	Count
lemma	171
def	69
instance	21
abbrev	4
theorem	1

2 Details by file

2.1 HepLean.FlavorPhysics.CKMMatrix.Basic

[HepLean/FlavorPhysics/CKMMatrix/Basic.lean](#) on GitHub.

lemma phaseShiftRelation_equiv

[\[source\]](#)

```
lemma phaseShiftRelation_equiv : Equivalence phaseShiftRelation where
```

2.2 HepLean.GroupTheory.S03.Basic

[HepLean/GroupTheory/S03/Basic.lean](#) on GitHub.

def S03

[\[source\]](#)

*The group of 3×3 real matrices with determinant 1 and $A * A^T = 1$.*

```
def S03 : Type
```

instance S03Group

[\[source\]](#)

```
@[simps mul_coe one_coe inv div]
```

```
instance S03Group : Group S03 where
```

instance TopologicalSpace S03

[\[source\]](#)

SO3 has the subtype topology.

```
instance : TopologicalSpace S03
```

lemma coe_inv

[\[source\]](#)

```
lemma coe_inv (A : S03) : (A⁻¹).1 = A.1⁻¹
```

def toGL

[\[source\]](#)

The inclusion of $SO(3)$ into $GL(\text{Fin } 3) \mathbb{R}$.

```
def toGL : S0(3) →* GL (Fin 3) ℝ where
```

lemma subtype_val_eq_toGL[\[source\]](#)

```
lemma subtype_val_eq_toGL : (Subtype.val : S03 → Matrix (Fin 3) (Fin 3) ℝ) =
  Units.val ∘ toGL.toFun
```

lemma toGL_injective[\[source\]](#)

```
lemma toGL_injective : Function.Injective toGL
```

def toProd[\[source\]](#)

The inclusion of $S0(3)$ into the monoid of matrices times the opposite of the monoid of matrices.

```
@[simps!]
def toProd : S0(3) →* (Matrix (Fin 3) (Fin 3) ℝ) × (Matrix (Fin 3) (Fin 3) ℝ)mop
```

lemma toProd_eq_transpose[\[source\]](#)

```
lemma toProd_eq_transpose : toProd A = (A.1, (A.1T))
```

lemma toProd_injective[\[source\]](#)

```
lemma toProd_injective : Function.Injective toProd
```

lemma toProd_continuous[\[source\]](#)

```
lemma toProd_continuous : Continuous toProd
```

lemma toProd_embedding[\[source\]](#)

The embedding of $S0(3)$ into the monoid of matrices times the opposite of the monoid of matrices.

```
lemma toProd_embedding : Embedding toProd where
```

lemma toGL_embedding[\[source\]](#)

The embedding of $S0(3)$ into $GL (Fin 3) ℝ$.

```
lemma toGL_embedding : Embedding toGL.toFun where
```

instance TopologicalGroup S0(3)[\[source\]](#)

```
instance : TopologicalGroup S0(3)
```

lemma det_minus_id[\[source\]](#)

```
lemma det_minus_id (A : S0(3)) : det (A.1 - 1) = 0
```

lemma det_id_minus[\[source\]](#)

```
@[simp]
lemma det_id_minus (A : SO(3)) : det (1 - A.1) = 0
```

lemma one_in_spectrum[\[source\]](#)

```
@[simp]
lemma one_in_spectrum (A : SO(3)) : 1 ∈ spectrum ℝ (A.1)
```

def toEnd[\[source\]](#)

The endomorphism of EuclideanSpace ℝ (Fin 3) associated to a element of SO(3).

```
@[simps!]
def toEnd (A : SO(3)) : End ℝ (EuclideanSpace ℝ (Fin 3))
```

lemma one_is_eigenvalue[\[source\]](#)

```
lemma one_is_eigenvalue (A : SO(3)) : A.toEnd.HasEigenvalue 1
```

lemma exists_stationary_vec[\[source\]](#)

```
lemma exists_stationary_vec (A : SO(3)) :
  ∃ (v : EuclideanSpace ℝ (Fin 3)),
    Orthonormal ℝ ({0} : Set (Fin 3)).restrict (fun _ => v)
  ∧ A.toEnd v = v
```

lemma exists_basis_preserved[\[source\]](#)

```
lemma exists_basis_preserved (A : SO(3)) :
  ∃ (b : OrthonormalBasis (Fin 3) ℝ (EuclideanSpace ℝ (Fin 3))), A.toEnd (b 0) = b 0
```

2.3 HepLean.SpaceTime.Basic

[HepLean/SpaceTime/Basic.lean](#) on GitHub.

def spaceTime[\[source\]](#)

The space-time

```
def spaceTime : Type
```

instance AddCommMonoid spaceTime[\[source\]](#)

Give spacetime the structure of an additive commutative monoid.

```
instance : AddCommMonoid spaceTime
```

instance Module ℝ spaceTime[\[source\]](#)

Give spacetime the structure of a module over the reals.

```
instance : Module ℝ spaceTime
```

```
instance euclideanNormedAddCommGroup
```

[\[source\]](#)

```
instance euclideanNormedAddCommGroup : NormedAddCommGroup spaceTime
```

```
instance euclideanNormedSpace
```

[\[source\]](#)

```
instance euclideanNormedSpace : NormedSpace ℝ spaceTime
```

```
def space
```

[\[source\]](#)

The space part of spacetime.

```
@[simp]
```

```
def space (x : spaceTime) : EuclideanSpace ℝ (Fin 3)
```

```
def asSmoothManifold
```

[\[source\]](#)

The structure of a smooth manifold on spacetime.

```
def asSmoothManifold : ModelWithCorners ℝ spaceTime spaceTime
```

```
instance ChartedSpace spaceTime spaceTime
```

[\[source\]](#)

```
instance : ChartedSpace spaceTime spaceTime
```

```
def stdBasis
```

[\[source\]](#)

The standard basis for spacetime.

```
def stdBasis : Basis (Fin 4) ℝ spaceTime
```

```
lemma stdBasis_apply
```

[\[source\]](#)

```
lemma stdBasis_apply (μ v : Fin 4) : stdBasis μ v = if μ = v then 1 else 0
```

```
lemma stdBasis_not_eq
```

[\[source\]](#)

```
lemma stdBasis_not_eq {μ v : Fin 4} (h : μ ≠ v) : stdBasis μ v = 0
```

```
lemma stdBasis_0
```

[\[source\]](#)

```
lemma stdBasis_0 : stdBasis 0 = ![1, 0, 0, 0]
```

```
lemma stdBasis_1
```

[\[source\]](#)

```
lemma stdBasis_1 : stdBasis 1 = ![0, 1, 0, 0]
```

lemma stdBasis_2[\[source\]](#)

```
lemma stdBasis_2 : stdBasis 2 = ![0, 0, 1, 0]
```

lemma stdBasis_3[\[source\]](#)

```
lemma stdBasis_3 : stdBasis 3 = ![0, 0, 0, 1]
```

lemma stdBasis_mulVec[\[source\]](#)

```
lemma stdBasis_mulVec (μ v : Fin 4) (Λ : Matrix (Fin 4) (Fin 4) ℝ) :
  (Λ *v stdBasis μ) v = Λ v μ
```

lemma explicit[\[source\]](#)

```
lemma explicit (x : spaceTime) : x = ![x 0, x 1, x 2, x 3]
```

2.4 HepLean.SpaceTime.CliffordAlgebra

[HepLean/SpaceTime/CliffordAlgebra.lean](#) on GitHub.

def γ0[\[source\]](#)

The γ^0 gamma matrix in the Dirac representation.

```
def γ0 : Matrix (Fin 4) (Fin 4) ℂ
```

def γ1[\[source\]](#)

The γ^1 gamma matrix in the Dirac representation.

```
def γ1 : Matrix (Fin 4) (Fin 4) ℂ
```

def γ2[\[source\]](#)

The γ^2 gamma matrix in the Dirac representation.

```
def γ2 : Matrix (Fin 4) (Fin 4) ℂ
```

def γ3[\[source\]](#)

The γ^3 gamma matrix in the Dirac representation.

```
def γ3 : Matrix (Fin 4) (Fin 4) ℂ
```

def γ5[\[source\]](#)

The γ^5 gamma matrix in the Dirac representation.

```
def γ5 : Matrix (Fin 4) (Fin 4) ℂ
```

def γ [\[source\]](#)

The γ gamma matrices in the Dirac representation.

```
@[simp]
def  $\gamma$  : Fin 4 → Matrix (Fin 4) (Fin 4) ℂ
```

def γ Set [\[source\]](#)

The subset of $\text{Matrix (Fin 4) (Fin 4) } \mathbb{C}$ formed by the gamma matrices in the Dirac representation.

```
@[simp]
def  $\gamma$ Set : Set (Matrix (Fin 4) (Fin 4) ℂ)
```

lemma γ _in_ γ Set [\[source\]](#)

```
lemma  $\gamma$ _in_ $\gamma$ Set ( $\mu$  : Fin 4) :  $\gamma \mu \in \gamma$ Set
```

def diracAlgebra [\[source\]](#)

The algebra generated by the gamma matrices in the Dirac representation.

```
def diracAlgebra : Subalgebra ℝ (Matrix (Fin 4) (Fin 4) ℂ)
```

lemma γ Set_subset_diracAlgebra [\[source\]](#)

```
lemma  $\gamma$ Set_subset_diracAlgebra :  $\gamma$ Set  $\subseteq$  diracAlgebra
```

lemma γ _in_diracAlgebra [\[source\]](#)

```
lemma  $\gamma$ _in_diracAlgebra ( $\mu$  : Fin 4) :  $\gamma \mu \in \text{diracAlgebra}$ 
```

2.5 HepLean.SpaceTime.FourVelocity

[HepLean/SpaceTime/FourVelocity.lean](#) on GitHub.

def PreFourVelocity [\[source\]](#)

A spaceTime vector for which $\eta_{\text{Lin}} x x = 1$.

```
@[simp]
def PreFourVelocity : Set spaceTime
```

instance TopologicalSpace PreFourVelocity [\[source\]](#)

```
instance : TopologicalSpace PreFourVelocity
```

lemma mem_PreFourVelocity_iff [\[source\]](#)

```
lemma mem_PreFourVelocity_iff {x : spaceTime} : x ∈ PreFourVelocity  $\leftrightarrow$   $\eta_{\text{Lin}} x x = 1$ 
```


def neg [\[source\]](#)

The negative of a PreFourVelocity as a PreFourVelocity.

```
def neg (v : PreFourVelocity) : PreFourVelocity
```

lemma zero_sq [\[source\]](#)

```
lemma zero_sq (v : PreFourVelocity) : v.1 0 ^ 2 = 1 + ||v.1.space|| ^ 2
```

lemma zero_abs_ge_one [\[source\]](#)

```
lemma zero_abs_ge_one (v : PreFourVelocity) : 1 ≤ |v.1 0|
```

lemma zero_abs_gt_norm_space [\[source\]](#)

```
lemma zero_abs_gt_norm_space (v : PreFourVelocity) : ||v.1.space|| < |v.1 0|
```

lemma zero_abs_ge_norm_space [\[source\]](#)

```
lemma zero_abs_ge_norm_space (v : PreFourVelocity) : ||v.1.space|| ≤ |v.1 0|
```

lemma zero_le_minus_one_or_ge_one [\[source\]](#)

```
lemma zero_le_minus_one_or_ge_one (v : PreFourVelocity) : v.1 0 ≤ -1 ∨ 1 ≤ v.1 0
```

lemma zero_nonpos_iff [\[source\]](#)

```
lemma zero_nonpos_iff (v : PreFourVelocity) : v.1 0 ≤ 0 ↔ v.1 0 ≤ - 1
```

lemma zero_nonneg_iff [\[source\]](#)

```
lemma zero_nonneg_iff (v : PreFourVelocity) : 0 ≤ v.1 0 ↔ 1 ≤ v.1 0
```

def IsFourVelocity [\[source\]](#)

A PreFourVelocity is a FourVelocity if its time componenet is non-negative.

```
def IsFourVelocity (v : PreFourVelocity) : Prop
```

lemma IsFourVelocity_abs_zero [\[source\]](#)

```
lemma IsFourVelocity_abs_zero {v : PreFourVelocity} (hv : IsFourVelocity v) :  
  |v.1 0| = v.1 0
```

lemma not_IsFourVelocity_iff [\[source\]](#)

```
lemma not_IsFourVelocity_iff (v : PreFourVelocity) : ¬ IsFourVelocity v ↔ v.1 0 ≤ 0
```

lemma not_IsFourVelocity_iff_neg[\[source\]](#)

```
lemma not_IsFourVelocity_iff_neg {v : PreFourVelocity} : ¬ IsFourVelocity v ↔
  IsFourVelocity (neg v)
```

lemma zero_abs_mul_sub_norm_space[\[source\]](#)

```
lemma zero_abs_mul_sub_norm_space (v v' : PreFourVelocity) :
  0 ≤ |v.1 0| * |v'.1 0| - ||v.1.space|| * ||v'.1.space||
```

lemma euclid_norm_IsFourVelocity_IsFourVelocity[\[source\]](#)

```
lemma euclid_norm_IsFourVelocity_IsFourVelocity {v v' : PreFourVelocity}
  (h : IsFourVelocity v) (h' : IsFourVelocity v') :
  0 ≤ (v.1 0) * (v'.1 0) + ⟨⟨v.1.space, v'.1.space⟩⟩_ℝ
```

lemma euclid_norm_not_IsFourVelocity_not_IsFourVelocity[\[source\]](#)

```
lemma euclid_norm_not_IsFourVelocity_not_IsFourVelocity {v v' : PreFourVelocity}
  (h : ¬ IsFourVelocity v) (h' : ¬ IsFourVelocity v') :
  0 ≤ (v.1 0) * (v'.1 0) + ⟨⟨v.1.space, v'.1.space⟩⟩_ℝ
```

lemma euclid_norm_IsFourVelocity_not_IsFourVelocity[\[source\]](#)

```
lemma euclid_norm_IsFourVelocity_not_IsFourVelocity {v v' : PreFourVelocity}
  (h : IsFourVelocity v) (h' : ¬ IsFourVelocity v') :
  (v.1 0) * (v'.1 0) + ⟨⟨v.1.space, v'.1.space⟩⟩_ℝ ≤ 0
```

lemma euclid_norm_not_IsFourVelocity_IsFourVelocity[\[source\]](#)

```
lemma euclid_norm_not_IsFourVelocity_IsFourVelocity {v v' : PreFourVelocity}
  (h : ¬ IsFourVelocity v) (h' : IsFourVelocity v') :
  (v.1 0) * (v'.1 0) + ⟨⟨v.1.space, v'.1.space⟩⟩_ℝ ≤ 0
```

def FourVelocity[\[source\]](#)

The set of PreFourVelocity's which are four velocities.

```
def FourVelocity : Set PreFourVelocity
```

instance TopologicalSpace FourVelocity[\[source\]](#)

```
instance : TopologicalSpace FourVelocity
```

lemma mem_FourVelocity_iff[\[source\]](#)

```
lemma mem_FourVelocity_iff {x : PreFourVelocity} : x ∈ FourVelocity ↔ 0 ≤ x.1 0
```

lemma time_comp[\[source\]](#)

```
lemma time_comp (x : FourVelocity) : x.1.1 0 = √(1 + ||x.1.1.space|| ^ 2)
```

def zero [\[source\]](#)

The FourVelocity which has all space components zero.

```
def zero : FourVelocity
```

def pathFromZero [\[source\]](#)

A continuous path from the zero FourVelocity to any other.

```
noncomputable def pathFromZero (u : FourVelocity) : Path zero u where
```

lemma isPathConnected_FourVelocity [\[source\]](#)

```
lemma isPathConnected_FourVelocity : IsPathConnected (@Set.univ FourVelocity)
```

lemma η_pos [\[source\]](#)

```
lemma η_pos (u v : FourVelocity) : 0 < ηLin u v
```

lemma one_plus_ne_zero [\[source\]](#)

```
lemma one_plus_ne_zero (u v : FourVelocity) : 1 + ηLin u v ≠ 0
```

lemma η_continuous [\[source\]](#)

```
lemma η_continuous (u : spaceTime) : Continuous (fun (a : FourVelocity) => ηLin u a)
```

2.6 HepLean.SpaceTime.LorentzAlgebra.Basic

[HepLean/SpaceTime/LorentzAlgebra/Basic.lean](#) on GitHub.

def lorentzAlgebra [\[source\]](#)

The Lorentz algebra as a subalgebra of Matrix (Fin 4) (Fin 4) ℝ.

```
def lorentzAlgebra : LieSubalgebra ℝ (Matrix (Fin 4) (Fin 4) ℝ)
```

lemma transpose_eta [\[source\]](#)

```
lemma transpose_eta (A : lorentzAlgebra) : A.1T * η = - η * A.1
```

lemma mem_of_transpose_eta_eq_eta_mul_self [\[source\]](#)

```
lemma mem_of_transpose_eta_eq_eta_mul_self {A : Matrix (Fin 4) (Fin 4) ℝ}
  (h : AT * η = - η * A) : A ∈ lorentzAlgebra
```

lemma mem_iff [\[source\]](#)

```
lemma mem_iff {A : Matrix (Fin 4) (Fin 4) ℝ} : A ∈ lorentzAlgebra ↔
  AT * η = - η * A
```

lemma mem_iff'[\[source\]](#)

```
lemma mem_iff' (A : Matrix (Fin 4) (Fin 4) ℝ) : A ∈ lorentzAlgebra ↔ A = - η * AT * η
```

instance spaceTimeAsLieRingModule[\[source\]](#)

@[simps!]

```
instance spaceTimeAsLieRingModule : LieRingModule lorentzAlgebra spaceTime where
```

instance spaceTimeAsLieModule[\[source\]](#)

@[simps!]

```
instance spaceTimeAsLieModule : LieModule ℝ lorentzAlgebra spaceTime where
```

2.7 HepLean.SpaceTime.LorentzGroup.Basic

[HepLean/SpaceTime/LorentzGroup/Basic.lean](#) on GitHub.

def PreservesηLin[\[source\]](#)

*We say a matrix Λ preserves ηLin if for all x and y , $\eta\text{Lin} (\Lambda *_v x) (\Lambda *_v y) = \eta\text{Lin} x y$.*

```
def PreservesηLin (Λ : Matrix (Fin 4) (Fin 4) ℝ) : Prop
```

lemma iff_on_right[\[source\]](#)

```
lemma iff_on_right : PreservesηLin Λ ↔
  ∀ (x y : spaceTime), ηLin x ((η * ΛT * η * Λ) *_v y) = ηLin x y
```

lemma iff_matrix[\[source\]](#)

```
lemma iff_matrix : PreservesηLin Λ ↔ η * ΛT * η * Λ = 1
```

lemma iff_matrix'[\[source\]](#)

```
lemma iff_matrix' : PreservesηLin Λ ↔ Λ * (η * ΛT * η) = 1
```

lemma iff_transpose[\[source\]](#)

```
lemma iff_transpose : PreservesηLin Λ ↔ PreservesηLin ΛT
```

def liftGL[\[source\]](#)

The lift of a matrix which preserves ηLin to an invertible matrix.

```
def liftGL {Λ : Matrix (Fin 4) (Fin 4) ℝ} (h : PreservesηLin Λ) : GL (Fin 4) ℝ
```

lemma mul [\[source\]](#)

```
lemma mul {Λ Λ' : Matrix (Fin 4) (Fin 4) ℝ} (h : PreservesηLin Λ) (h' : PreservesηLin Λ') :
  PreservesηLin (Λ * Λ')
```

lemma one [\[source\]](#)

```
lemma one : PreservesηLin 1
```

lemma η [\[source\]](#)

```
lemma η : PreservesηLin η
```

def lorentzGroup [\[source\]](#)

The Lorentz group is the subset of matrices which preserve η Lin.

```
def lorentzGroup : Type
```

instance lorentzGroupIsGroup [\[source\]](#)

```
@[simps mul_coe one_coe inv div]
instance lorentzGroupIsGroup : Group lorentzGroup where
```

instance TopologicalSpace lorentzGroup [\[source\]](#)

lorentzGroup has the subtype topology.

```
instance : TopologicalSpace lorentzGroup
```

lemma coe_inv [\[source\]](#)

```
lemma coe_inv (A : ℒ) : (A-1).1 = A.1-1
```

def toGL [\[source\]](#)

The homomorphism of the Lorentz group into GL (Fin 4) ℝ.

```
def toGL : ℒ →* GL (Fin 4) ℝ where
```

lemma toGL_injective [\[source\]](#)

```
lemma toGL_injective : Function.Injective toGL
```

def toProd [\[source\]](#)

The homomorphism from the Lorentz Group into the monoid of matrices times the opposite of the monoid of matrices.

```
@[simps!]
def toProd : ℒ →* (Matrix (Fin 4) (Fin 4) ℝ) × (Matrix (Fin 4) (Fin 4) ℝ)mop
```

lemma toProd_eq_transpose_η[\[source\]](#)

```
lemma toProd_eq_transpose_η : toProd A = (A.1, (η * A.1T * η))
```

lemma toProd_injective[\[source\]](#)

```
lemma toProd_injective : Function.Injective toProd
```

lemma toProd_continuous[\[source\]](#)

```
lemma toProd_continuous : Continuous toProd
```

lemma toProd_embedding[\[source\]](#)

The embedding from the Lorentz Group into the monoid of matrices times the opposite of the monoid of matrices.

```
lemma toProd_embedding : Embedding toProd where
```

lemma toGL_embedding[\[source\]](#)

The embedding from the Lorentz Group into GL (Fin 4) ℝ.

```
lemma toGL_embedding : Embedding toGL.toFun where
```

instance TopologicalGroup $\mathcal{L}$ [\[source\]](#)

```
instance : TopologicalGroup  $\mathcal{L}$ 
```

def transpose[\[source\]](#)

The transpose of an matrix in the Lorentz group is an element of the Lorentz group.

```
def transpose (Λ : lorentzGroup) : lorentzGroup
```

2.8 HepLean.SpaceTime.LorentzGroup.Boosts

[HepLean/SpaceTime/LorentzGroup/Boosts.lean](#) on GitHub.

def genBoostAux₁[\[source\]](#)

An auxillary linear map used in the definition of a gearlised boost.

```
def genBoostAux1 (u v : FourVelocity) : spaceTime →l ℝ spaceTime where
```

def genBoostAux₂[\[source\]](#)

An auxillary linear map used in the definition of a gearlised boost.

```
def genBoostAux2 (u v : FourVelocity) : spaceTime →l ℝ spaceTime where
```

def genBoost [\[source\]](#)

An genearlised boost. This is a Lorentz transformation which takes the four velocity u to v .

```
def genBoost (u v : FourVelocity) : spaceTime →l[ℝ] spaceTime
```

lemma self [\[source\]](#)

```
lemma self (u : FourVelocity) : genBoost u u = LinearMap.id
```

def toMatrix [\[source\]](#)

A generalised boost as a matrix.

```
def toMatrix (u v : FourVelocity) : Matrix (Fin 4) (Fin 4) ℝ
```

lemma toMatrix_mulVec [\[source\]](#)

```
lemma toMatrix_mulVec (u v : FourVelocity) (x : spaceTime) :  
  (toMatrix u v).mulVec x = genBoost u v x
```

lemma toMatrix_apply [\[source\]](#)

```
lemma toMatrix_apply (u v : FourVelocity) (i j : Fin 4) :  
  (toMatrix u v) i j =  
    η i i * (ηLin (stdBasis i) (stdBasis j) + 2 * ηLin (stdBasis j) u * ηLin (stdBasis i) v -  
    ηLin (stdBasis i) (u + v) * ηLin (stdBasis j) (u + v) / (1 + ηLin u v))
```

lemma toMatrix_continuous [\[source\]](#)

```
lemma toMatrix_continuous (u : FourVelocity) : Continuous (toMatrix u)
```

lemma toMatrix_PreservesηLin [\[source\]](#)

```
lemma toMatrix_PreservesηLin (u v : FourVelocity) : PreservesηLin (toMatrix u v)
```

def toLorentz [\[source\]](#)

A generalised boost as an element of the Lorentz Group.

```
def toLorentz (u v : FourVelocity) : lorentzGroup
```

lemma toLorentz_continuous [\[source\]](#)

```
lemma toLorentz_continuous (u : FourVelocity) : Continuous (toLorentz u)
```

lemma toLorentz_joined_to_1 [\[source\]](#)

```
lemma toLorentz_joined_to_1 (u v : FourVelocity) : Joined 1 (toLorentz u v)
```

lemma toLorentz_in_connected_component_1[\[source\]](#)

```
lemma toLorentz_in_connected_component_1 (u v : FourVelocity) :
  toLorentz u v ∈ connectedComponent 1
```

lemma isProper[\[source\]](#)

```
lemma isProper (u v : FourVelocity) : IsProper (toLorentz u v)
```

2.9 HepLean.SpaceTime.LorentzGroup.Orthochronous

[HepLean/SpaceTime/LorentzGroup/Orthochronous.lean](#) on GitHub.

def fstCol[\[source\]](#)

The first column of a lorentz matrix as a PreFourVelocity.

```
@[simp]
def fstCol (Λ : lorentzGroup) : PreFourVelocity
```

def IsOrthochronous[\[source\]](#)

A Lorentz transformation is orthochronous if its $\theta \ \theta$ element is non-negative.

```
def IsOrthochronous (Λ : lorentzGroup) : Prop
```

lemma IsOrthochronous_iff_transpose[\[source\]](#)

```
lemma IsOrthochronous_iff_transpose (Λ : lorentzGroup) :
  IsOrthochronous Λ ↔ IsOrthochronous (transpose Λ)
```

lemma IsOrthochronous_iff_fstCol_IsFourVelocity[\[source\]](#)

```
lemma IsOrthochronous_iff_fstCol_IsFourVelocity (Λ : lorentzGroup) :
  IsOrthochronous Λ ↔ IsFourVelocity (fstCol Λ)
```

def mapZeroZeroComp[\[source\]](#)

The continuous map taking a Lorentz transformation to its $\theta \ \theta$ element.

```
def mapZeroZeroComp : C(lorentzGroup, ℝ)
```

def stepFunction[\[source\]](#)

An auxillary function used in the definition of orthchroMapReal.

```
def stepFunction : ℝ → ℝ
```

lemma stepFunction_continuous[\[source\]](#)

```
lemma stepFunction_continuous : Continuous stepFunction
```


def orthchroMapReal[\[source\]](#)

The continuous map from `lorentzGroup` to $\mathbb{R}$ wh taking Orthochronous elements to 1 and non-orthochronous to -1.

```
def orthchroMapReal : C(lorentzGroup, ℝ)
```

lemma orthchroMapReal_on_IsOrthochronous[\[source\]](#)

```
lemma orthchroMapReal_on_IsOrthochronous {Λ : lorentzGroup} (h : IsOrthochronous Λ) :
  orthchroMapReal Λ = 1
```

lemma orthchroMapReal_on_not_IsOrthochronous[\[source\]](#)

```
lemma orthchroMapReal_on_not_IsOrthochronous {Λ : lorentzGroup} (h : ¬ IsOrthochronous Λ) :
  orthchroMapReal Λ = - 1
```

lemma orthchroMapReal_minus_one_or_one[\[source\]](#)

```
lemma orthchroMapReal_minus_one_or_one (Λ : lorentzGroup) :
  orthchroMapReal Λ = -1 ∨ orthchroMapReal Λ = 1
```

def orthchroMap[\[source\]](#)

A continuous map from `lorentzGroup` to $\mathbb{Z}_2$ whose kernal are the Orthochronous elements.

```
def orthchroMap : C(lorentzGroup, ℤ₂)
```

lemma orthchroMap_IsOrthochronous[\[source\]](#)

```
lemma orthchroMap_IsOrthochronous {Λ : lorentzGroup} (h : IsOrthochronous Λ) :
  orthchroMap Λ = 1
```

lemma orthchroMap_not_IsOrthochronous[\[source\]](#)

```
lemma orthchroMap_not_IsOrthochronous {Λ : lorentzGroup} (h : ¬ IsOrthochronous Λ) :
  orthchroMap Λ = Additive.toMul (1 : ZMod 2)
```

lemma zero_zero_mul[\[source\]](#)

```
lemma zero_zero_mul (Λ Λ' : lorentzGroup) :
  (Λ * Λ').1 0 0 = (fstCol (transpose Λ)).1 0 * (fstCol Λ').1 0 +
  ⟨⟨fstCol (transpose Λ)).1.space, (fstCol Λ').1.space⟩⟩_ℝ
```

lemma mul_othchron_of_othchron_othchron[\[source\]](#)

```
lemma mul_othchron_of_othchron_othchron {Λ Λ' : lorentzGroup} (h : IsOrthochronous Λ)
  (h' : IsOrthochronous Λ') : IsOrthochronous (Λ * Λ')
```

lemma mul_othchron_of_not_othchron_not_othchron[\[source\]](#)

```
lemma mul_othchron_of_not_othchron_not_othchron {Λ Λ' : lorentzGroup} (h : ¬ IsOrthochronous
  Λ)
  (h' : ¬ IsOrthochronous Λ') : IsOrthochronous (Λ * Λ')
```

lemma mul_not_othchron_of_othchron_not_othchron[\[source\]](#)

```
lemma mul_not_othchron_of_othchron_not_othchron {Λ Λ' : lorentzGroup} (h : IsOrthochronous Λ)
  (h' : ¬ IsOrthochronous Λ') : ¬ IsOrthochronous (Λ * Λ')
```

lemma mul_not_othchron_of_not_othchron_othchron[\[source\]](#)

```
lemma mul_not_othchron_of_not_othchron_othchron {Λ Λ' : lorentzGroup} (h : ¬ IsOrthochronous Λ)
  (h' : IsOrthochronous Λ') : ¬ IsOrthochronous (Λ * Λ')
```

def orthchroRep[\[source\]](#)

The homomorphism from lorentzGroup to $\mathbb{Z}_2$ whose kernel are the Orthochronous elements.

```
def orthchroRep : lorentzGroup →* ℤ₂ where
```

2.10 HepLean.SpaceTime.LorentzGroup.Proper

[HepLean/SpaceTime/LorentzGroup/Proper.lean](#) on GitHub.

lemma det_eq_one_or_neg_one[\[source\]](#)

The determinant of a member of the lorentz group is 1 or -1.

```
lemma det_eq_one_or_neg_one (Λ : ℒ) : Λ.1.det = 1 ∨ Λ.1.det = -1
```

instance TopologicalSpace ℤ₂[\[source\]](#)

```
instance : TopologicalSpace ℤ₂
```

instance DiscreteTopology ℤ₂[\[source\]](#)

```
instance : DiscreteTopology ℤ₂
```

instance TopologicalGroup ℤ₂[\[source\]](#)

```
instance : TopologicalGroup ℤ₂
```

def coeForℤ₂[\[source\]](#)

A continuous function from $(\{-1, 1\} : \text{Set } \mathbb{R})$ to $\mathbb{Z}_2$.

```
@[simps!]
def coeForℤ₂ : C(({-1, 1} : Set ℝ), ℤ₂) where
```

def detContinuous[\[source\]](#)

The continuous map taking a lorentz matrix to its determinant.

```
def detContinuous : C( $\mathcal{L}$ ,  $\mathbb{Z}_2$ )
```

lemma detContinuous_eq_iff_det_eq[\[source\]](#)

```
lemma detContinuous_eq_iff_det_eq ( $\Lambda \Lambda'$  : lorentzGroup) :  
  detContinuous  $\Lambda$  = detContinuous  $\Lambda'$   $\leftrightarrow$   $\Lambda.1.\text{det}$  =  $\Lambda'.1.\text{det}$ 
```

def detRep[\[source\]](#)

The representation taking a lorentz matrix to its determinant.

```
@[simps!]  
def detRep :  $\mathcal{L} \rightarrow^* \mathbb{Z}_2$  where
```

lemma detRep_continuous[\[source\]](#)

```
lemma detRep_continuous : Continuous detRep
```

lemma det_on_connected_component[\[source\]](#)

```
lemma det_on_connected_component { $\Lambda \Lambda'$  : lorentzGroup} (h :  $\Lambda' \in \text{connectedComponent } \Lambda$ ) :  
   $\Lambda.1.\text{det}$  =  $\Lambda'.1.\text{det}$ 
```

lemma detRep_on_connected_component[\[source\]](#)

```
lemma detRep_on_connected_component { $\Lambda \Lambda'$  : lorentzGroup} (h :  $\Lambda' \in \text{connectedComponent } \Lambda$ ) :  
  detRep  $\Lambda$  = detRep  $\Lambda'$ 
```

lemma det_of_joined[\[source\]](#)

```
lemma det_of_joined { $\Lambda \Lambda'$  : lorentzGroup} (h : Joined  $\Lambda \Lambda'$ ) :  $\Lambda.1.\text{det}$  =  $\Lambda'.1.\text{det}$ 
```

def IsProper[\[source\]](#)

A Lorentz Matrix is proper if its determinant is 1.

```
@[simp]  
def IsProper ( $\Lambda$  : lorentzGroup) : Prop
```

instance DecidablePred IsProper[\[source\]](#)

```
instance : DecidablePred IsProper
```

lemma IsProper_iff[\[source\]](#)

```
lemma IsProper_iff ( $\Lambda$  : lorentzGroup) : IsProper  $\Lambda \leftrightarrow$  detRep  $\Lambda$  = 1
```

lemma id_IsProper[\[source\]](#)

```
lemma id_IsProper : IsProper 1
```

lemma isProper_on_connected_component[\[source\]](#)

```
lemma isProper_on_connected_component {Λ Λ' : lorentzGroup} (h : Λ' ∈ connectedComponent Λ) :
  IsProper Λ ↔ IsProper Λ'
```

2.11 HepLean.SpaceTime.LorentzGroup.Rotations

[HepLean/SpaceTime/LorentzGroup/Rotations.lean](#) on GitHub.

def S03ToMatrix[\[source\]](#)

Given a element of $SO(3)$ the matrix corresponding to a space-time rotation.

```
@[simp]
def S03ToMatrix (A : SO(3)) : Matrix (Fin 4) (Fin 4) ℝ
```

lemma S03ToMatrix_PreservesηLin[\[source\]](#)

```
lemma S03ToMatrix_PreservesηLin (A : SO(3)) : PreservesηLin $ S03ToMatrix A
```

lemma S03ToMatrix_injective[\[source\]](#)

```
lemma S03ToMatrix_injective : Function.Injective S03ToMatrix
```

def S03ToLorentz[\[source\]](#)

Given a element of $SO(3)$ the element of the Lorentz group corresponding to a space-time rotation.

```
def S03ToLorentz : SO(3) →* ℒ where
```

2.12 HepLean.SpaceTime.Metric

[HepLean/SpaceTime/Metric.lean](#) on GitHub.

def η[\[source\]](#)

The metric as a 4×4 real matrix.

```
def η : Matrix (Fin 4) (Fin 4) ℝ
```

lemma η_block[\[source\]](#)

```
lemma η_block : η = Matrix.reindex finSumFinEquiv finSumFinEquiv (
  Matrix.fromBlocks (1 : Matrix (Fin 1) (Fin 1) ℝ) 0 0 (-1 : Matrix (Fin 3) (Fin 3) ℝ))
```

lemma η_{reindex} [\[source\]](#)

```
lemma  $\eta_{\text{reindex}}$  : (Matrix.reindex finSumFinEquiv finSumFinEquiv).symm  $\eta$  =
  LieAlgebra.Orthogonal.indefiniteDiagonal (Fin 1) (Fin 3)  $\mathbb{R}$ 
```

lemma η_{explicit} [\[source\]](#)

```
lemma  $\eta_{\text{explicit}}$  :  $\eta$  = !![(1 :  $\mathbb{R}$ ), 0, 0, 0; 0, -1, 0, 0; 0, 0, -1, 0; 0, 0, 0, -1]
```

lemma $\eta_{\text{off_diagonal}}$ [\[source\]](#)

```
@[simp]
lemma  $\eta_{\text{off\_diagonal}}$  { $\mu$   $\nu$  : Fin 4} (h :  $\mu \neq \nu$ ) :  $\eta$   $\mu$   $\nu$  = 0
```

lemma $\eta_{\text{symmetric}}$ [\[source\]](#)

```
lemma  $\eta_{\text{symmetric}}$  ( $\mu$   $\nu$  : Fin 4) :  $\eta$   $\mu$   $\nu$  =  $\eta$   $\nu$   $\mu$ 
```

lemma $\eta_{\text{transpose}}$ [\[source\]](#)

```
@[simp]
lemma  $\eta_{\text{transpose}}$  :  $\eta$ .transpose =  $\eta$ 
```

lemma $\det_{\eta}$ [\[source\]](#)

```
@[simp]
lemma  $\det_{\eta}$  :  $\eta$ .det = - 1
```

lemma η_{sq} [\[source\]](#)

```
@[simp]
lemma  $\eta_{\text{sq}}$  :  $\eta$  *  $\eta$  = 1
```

lemma $\eta_{\text{diag_mul_self}}$ [\[source\]](#)

```
lemma  $\eta_{\text{diag\_mul\_self}}$  ( $\mu$  : Fin 4) :  $\eta$   $\mu$   $\mu$  *  $\eta$   $\mu$   $\mu$  = 1
```

lemma η_{mulVec} [\[source\]](#)

```
lemma  $\eta_{\text{mulVec}}$  (x : spaceTime) :  $\eta$  *v x = ![x 0, -x 1, -x 2, -x 3]
```

def linearMapForSpaceTime[\[source\]](#)

*Given a point in spaceTime x the linear map $y \rightarrow x \cdot_v (\eta *_{\nu} y)$.*

```
@[simpls!]
def linearMapForSpaceTime (x : spaceTime) : spaceTime  $\rightarrow_l$  [ $\mathbb{R}$ ]  $\mathbb{R}$  where
```

def ηLin [\[source\]](#)

The metric as a bilinear map from spaceTime to $\mathbb{R}$.

def ηLin : LinearMap.BilinForm $\mathbb{R}$ spaceTime **where**

lemma $\eta\text{Lin_expand}$ [\[source\]](#)

lemma $\eta\text{Lin_expand}$ ($x\ y$: spaceTime) : $\eta\text{Lin}\ x\ y = x\ 0 * y\ 0 - x\ 1 * y\ 1 - x\ 2 * y\ 2 - x\ 3 * y\ 3$

lemma $\eta\text{Lin_expand_self}$ [\[source\]](#)

lemma $\eta\text{Lin_expand_self}$ (x : spaceTime) : $\eta\text{Lin}\ x\ x = x\ 0^2 - \|x.\text{space}\|^2$

lemma $\text{time_elm_sq_of_}\eta\text{Lin}$ [\[source\]](#)

lemma $\text{time_elm_sq_of_}\eta\text{Lin}$ (x : spaceTime) : $x\ 0^2 = \eta\text{Lin}\ x\ x + \|x.\text{space}\|^2$

lemma $\eta\text{Lin_leq_time_sq}$ [\[source\]](#)

lemma $\eta\text{Lin_leq_time_sq}$ (x : spaceTime) : $\eta\text{Lin}\ x\ x \leq x\ 0^2$

lemma $\eta\text{Lin_space_inner_product}$ [\[source\]](#)

lemma $\eta\text{Lin_space_inner_product}$ ($x\ y$: spaceTime) :
 $\eta\text{Lin}\ x\ y = x\ 0 * y\ 0 - \langle\langle x.\text{space}, y.\text{space} \rangle\rangle_{\mathbb{R}}$

lemma $\eta\text{Lin_ge_abs_inner_product}$ [\[source\]](#)

lemma $\eta\text{Lin_ge_abs_inner_product}$ ($x\ y$: spaceTime) :
 $x\ 0 * y\ 0 - \|\langle\langle x.\text{space}, y.\text{space} \rangle\rangle_{\mathbb{R}}\| \leq (\eta\text{Lin}\ x\ y)$

lemma $\eta\text{Lin_ge_sub_norm}$ [\[source\]](#)

lemma $\eta\text{Lin_ge_sub_norm}$ ($x\ y$: spaceTime) :
 $x\ 0 * y\ 0 - \|x.\text{space}\| * \|y.\text{space}\| \leq (\eta\text{Lin}\ x\ y)$

lemma $\eta\text{Lin_symm}$ [\[source\]](#)

lemma $\eta\text{Lin_symm}$ ($x\ y$: spaceTime) : $\eta\text{Lin}\ x\ y = \eta\text{Lin}\ y\ x$

lemma $\eta\text{Lin_stdBasis_apply}$ [\[source\]](#)

lemma $\eta\text{Lin_stdBasis_apply}$ (μ : Fin 4) (x : spaceTime) : $\eta\text{Lin}\ (\text{stdBasis}\ \mu)\ x = \eta\ \mu\ \mu * x\ \mu$

lemma $\eta\text{Lin_}\eta\text{_stdBasis}$ [\[source\]](#)

lemma $\eta\text{Lin_}\eta\text{_stdBasis}$ ($\mu\ v$: Fin 4) : $\eta\text{Lin}\ (\text{stdBasis}\ \mu)\ (\text{stdBasis}\ v) = \eta\ \mu\ v$

lemma $\eta\text{Lin_mulVec_left}$ [\[source\]](#)

```
lemma  $\eta\text{Lin\_mulVec\_left}$  (x y : spaceTime) ( $\Lambda$  : Matrix (Fin 4) (Fin 4)  $\mathbb{R}$ ) :
   $\eta\text{Lin}$  ( $\Lambda *_{\nu}$  x) y =  $\eta\text{Lin}$  x (( $\eta * \Lambda^T * \eta$ )  $*_{\nu}$  y)
```

lemma $\eta\text{Lin_mulVec_right}$ [\[source\]](#)

```
lemma  $\eta\text{Lin\_mulVec\_right}$  (x y : spaceTime) ( $\Lambda$  : Matrix (Fin 4) (Fin 4)  $\mathbb{R}$ ) :
   $\eta\text{Lin}$  x ( $\Lambda *_{\nu}$  y) =  $\eta\text{Lin}$  (( $\eta * \Lambda^T * \eta$ )  $*_{\nu}$  x) y
```

lemma $\eta\text{Lin_matrix_stdBasis}$ [\[source\]](#)

```
lemma  $\eta\text{Lin\_matrix\_stdBasis}$  ( $\mu \nu$  : Fin 4) ( $\Lambda$  : Matrix (Fin 4) (Fin 4)  $\mathbb{R}$ ) :
   $\eta\text{Lin}$  (stdBasis  $\nu$ ) ( $\Lambda *_{\nu}$  stdBasis  $\mu$ ) =  $\eta \nu \nu * \Lambda \nu \mu$ 
```

lemma $\eta\text{Lin_matrix_stdBasis}'$ [\[source\]](#)

```
lemma  $\eta\text{Lin\_matrix\_stdBasis}'$  ( $\mu \nu$  : Fin 4) ( $\Lambda$  : Matrix (Fin 4) (Fin 4)  $\mathbb{R}$ ) :
   $\Lambda \nu \mu$  =  $\eta \nu \nu * \eta\text{Lin}$  (stdBasis  $\nu$ ) ( $\Lambda *_{\nu}$  stdBasis  $\mu$ )
```

lemma $\eta\text{Lin_matrix_eq_identity_iff}$ [\[source\]](#)

```
lemma  $\eta\text{Lin\_matrix\_eq\_identity\_iff}$  ( $\Lambda$  : Matrix (Fin 4) (Fin 4)  $\mathbb{R}$ ) :
   $\Lambda = 1 \leftrightarrow \forall$  (x y : spaceTime),  $\eta\text{Lin}$  x y =  $\eta\text{Lin}$  x ( $\Lambda *_{\nu}$  y)
```

def quadraticForm[\[source\]](#)

The metric as a quadratic form on spaceTime.

```
def quadraticForm : QuadraticForm  $\mathbb{R}$  spaceTime
```

2.13 HepLean.StandardModel.Basic

[HepLean/StandardModel/Basic.lean](#) on GitHub.

abbrev gaugeGroup[\[source\]](#)

The global gauge group of the standard model. TODO: Generalize to quotient.

```
abbrev gaugeGroup : Type
```

2.14 HepLean.StandardModel.HiggsBoson.Basic

[HepLean/StandardModel/HiggsBoson/Basic.lean](#) on GitHub.

abbrev higgsBundle[\[source\]](#)

The trivial vector bundle $\mathcal{R}^2 \times \mathbb{C}^2$. (TODO: Make associated bundle.)

```
abbrev higgsBundle
```

instance SmoothVectorBundle higgsVec higgsBundle spaceTime.asSmoothManifold [\[source\]](#)

instance : SmoothVectorBundle higgsVec higgsBundle spaceTime.asSmoothManifold

abbrev higgsField [\[source\]](#)

A higgs field is a smooth section of the higgs bundle.

abbrev higgsField : Type

instance NormedAddCommGroup (Fin 2 → ℂ) [\[source\]](#)

instance : NormedAddCommGroup (Fin 2 → ℂ)

def toField [\[source\]](#)

Given a vector $\mathbb{C}^2$ the constant higgs field with value equal to that section.

noncomputable def higgsVec.toField (ϕ : higgsVec) : higgsField **where**

def toHiggsVec [\[source\]](#)

Given a higgsField, the corresponding map from spaceTime to higgsVec.

def toHiggsVec (ϕ : higgsField) : spaceTime → higgsVec

lemma toHiggsVec_smooth [\[source\]](#)

lemma toHiggsVec_smooth (ϕ : higgsField) : Smooth $\mathcal{I}(\mathbb{R}, \text{spaceTime}) \mathcal{I}(\mathbb{R}, \text{higgsVec}) \phi.\text{toHiggsVec}$

lemma toField_toHiggsVec_apply [\[source\]](#)

lemma toField_toHiggsVec_apply (ϕ : higgsField) (x : spaceTime) :
($\phi.\text{toHiggsVec } x$).toField $x = \phi x$

lemma higgsVecToFin2ℂ_toHiggsVec [\[source\]](#)

lemma higgsVecToFin2ℂ_toHiggsVec (ϕ : higgsField) : higgsVecToFin2ℂ ∘ $\phi.\text{toHiggsVec} = \phi$

lemma toVec_smooth [\[source\]](#)

lemma toVec_smooth (ϕ : higgsField) : Smooth $\mathcal{I}(\mathbb{R}, \text{spaceTime}) \mathcal{I}(\mathbb{R}, \text{Fin } 2 \rightarrow \mathbb{C}) \phi$

lemma apply_smooth [\[source\]](#)

lemma apply_smooth (ϕ : higgsField) :
 $\forall i, \text{Smooth } \mathcal{I}(\mathbb{R}, \text{spaceTime}) \mathcal{I}(\mathbb{R}, \mathbb{C})$ (**fun** (x : spaceTime) => ($\phi x i$))

lemma apply_re_smooth[\[source\]](#)

```
lemma apply_re_smooth (φ : higgsField) (i : Fin 2):
  Smooth  $\mathcal{I}(\mathbb{R}, \text{spaceTime})$   $\mathcal{I}(\mathbb{R}, \mathbb{R})$  (reCLM ∘ (fun (x : spaceTime) => (φ x i)))
```

lemma apply_im_smooth[\[source\]](#)

```
lemma apply_im_smooth (φ : higgsField) (i : Fin 2):
  Smooth  $\mathcal{I}(\mathbb{R}, \text{spaceTime})$   $\mathcal{I}(\mathbb{R}, \mathbb{R})$  (imCLM ∘ (fun (x : spaceTime) => (φ x i)))
```

def normSq[\[source\]](#)

Given a higgsField, the map $\text{spaceTime} \rightarrow \mathbb{R}$ obtained by taking the square norm of the higgs vector.

```
@[simp]
def normSq (φ : higgsField) : spaceTime → ℝ
```

lemma toHiggsVec_norm[\[source\]](#)

```
lemma toHiggsVec_norm (φ : higgsField) (x : spaceTime) :
  ||φ x|| = ||φ.toHiggsVec x||
```

lemma normSq_expand[\[source\]](#)

```
lemma normSq_expand (φ : higgsField) :
  φ.normSq = fun x => (conj (φ x 0) * (φ x 0) + conj (φ x 1) * (φ x 1)).re
```

lemma normSq_smooth[\[source\]](#)

```
lemma normSq_smooth (φ : higgsField) : Smooth  $\mathcal{I}(\mathbb{R}, \text{spaceTime})$   $\mathcal{I}(\mathbb{R}, \mathbb{R})$  φ.normSq
```

lemma normSq_nonneg[\[source\]](#)

```
lemma normSq_nonneg (φ : higgsField) (x : spaceTime) : 0 ≤ φ.normSq x
```

lemma normSq_zero[\[source\]](#)

```
lemma normSq_zero (φ : higgsField) (x : spaceTime) : φ.normSq x = 0 ↔ φ x = 0
```

def potential[\[source\]](#)

*The Higgs potential of the form $-\mu^2 * |\varphi|^2 + \lambda * |\varphi|^4$.*

```
@[simp]
def potential (φ : higgsField) (μSq lambda : ℝ) (x : spaceTime) : ℝ
```

lemma potential_smooth[\[source\]](#)

```
lemma potential_smooth (φ : higgsField) (μSq lambda : ℝ) :
  Smooth  $\mathcal{I}(\mathbb{R}, \text{spaceTime})$   $\mathcal{I}(\mathbb{R}, \mathbb{R})$  (fun x => φ.potential μSq lambda x)
```

lemma potential_apply[\[source\]](#)

```
lemma potential_apply (φ : higgsField) (μSq lambda : ℝ) (x : spaceTime) :
  (φ.potential μSq lambda) x = higgsVec.potential μSq lambda (φ.toHiggsVec x)
```

def isConst[\[source\]](#)

A higgs field is constant if it is equal for all x y in $spaceTime$.

```
def isConst (Φ : higgsField) : Prop
```

lemma isConst_of_higgsVec[\[source\]](#)

```
lemma isConst_of_higgsVec (φ : higgsVec) : φ.toField.isConst
```

lemma isConst_iff_of_higgsVec[\[source\]](#)

```
lemma isConst_iff_of_higgsVec (Φ : higgsField) : Φ.isConst ↔ ∃ (φ : higgsVec), Φ = φ.toField
```

lemma normSq_of_higgsVec[\[source\]](#)

```
lemma normSq_of_higgsVec (φ : higgsVec) : φ.toField.normSq = fun x => (norm φ) ^ 2
```

lemma potential_of_higgsVec[\[source\]](#)

```
lemma potential_of_higgsVec (φ : higgsVec) (μSq lambda : ℝ) :
  φ.toField.potential μSq lambda = fun _ => higgsVec.potential μSq lambda φ
```

2.15 HepLean.StandardModel.HiggsBoson.TargetSpace

[HepLean/StandardModel/HiggsBoson/TargetSpace.lean](#) on GitHub.

abbrev higgsVec[\[source\]](#)

The complex vector space in which the Higgs field takes values.

```
abbrev higgsVec
```

def higgsVecToFin2C[\[source\]](#)

The continuous linear map from the vector space $higgsVec$ to $(Fin\ 2 \rightarrow \mathbb{C})$ achieved by casting vectors.

```
def higgsVecToFin2C : higgsVec →L[ℝ] (Fin 2 → ℂ) where
```

lemma smooth_higgsVecToFin2C[\[source\]](#)

```
lemma smooth_higgsVecToFin2C : Smooth ℐ(ℝ, higgsVec) ℐ(ℝ, Fin 2 → ℂ) higgsVecToFin2C
```

def higgsRepUnitary[\[source\]](#)

The Higgs representation as a homomorphism from the gauge group to unitary 2×2 -matrices.

@[simps!]

```
noncomputable def higgsRepUnitary : gaugeGroup →* unitaryGroup (Fin 2) ℂ where
```

def orthonormBasis[\[source\]](#)

An orthonormal basis of higgsVec.

```
noncomputable def orthonormBasis : OrthonormalBasis (Fin 2) ℂ higgsVec
```

def matrixToLin[\[source\]](#)

Takes in a 2×2 -matrix and returns a linear map of higgsVec.

```
noncomputable def matrixToLin : Matrix (Fin 2) (Fin 2) ℂ →* (higgsVec →L[ℂ] higgsVec) where
```

lemma matrixToLin_star[\[source\]](#)

```
lemma matrixToLin_star (g : Matrix (Fin 2) (Fin 2) ℂ) :  
  matrixToLin (star g) = star (matrixToLin g)
```

lemma matrixToLin_unitary[\[source\]](#)

```
lemma matrixToLin_unitary (g : unitaryGroup (Fin 2) ℂ) :  
  matrixToLin g ∈ unitary (higgsVec →L[ℂ] higgsVec)
```

def unitaryToLin[\[source\]](#)

The natural homomorphism from unitary 2×2 complex matrices to unitary transformations of higgsVec.

```
noncomputable def unitaryToLin : unitaryGroup (Fin 2) ℂ →* unitary (higgsVec →L[ℂ] higgsVec)  
  where
```

def unitToLinear[\[source\]](#)

The inclusion of unitary transformations on higgsVec into all linear transformations.

@[simps!]

```
def unitToLinear : unitary (higgsVec →L[ℂ] higgsVec) →* higgsVec →L[ℂ] higgsVec
```

def rep[\[source\]](#)

The representation of the gauge group acting on higgsVec.

@[simps!]

```
def rep : Representation ℂ gaugeGroup higgsVec
```

lemma higgsRepUnitary_mul[\[source\]](#)

```
lemma higgsRepUnitary_mul (g : gaugeGroup) (φ : higgsVec) :
  (higgsRepUnitary g).1 *v φ = g.2.2 ^ 3 • (g.2.1.1 *v φ)
```

lemma rep_apply[\[source\]](#)

```
lemma rep_apply (g : gaugeGroup) (φ : higgsVec) : rep g φ = g.2.2 ^ 3 • (g.2.1.1 *v φ)
```

lemma norm_invariant[\[source\]](#)

```
lemma norm_invariant (g : gaugeGroup) (φ : higgsVec) : ||rep g φ|| = ||φ||
```

def potential[\[source\]](#)

The higgs potential for higgsVec, i.e. for constant higgs fields.

```
def potential (φ : higgsVec) : ℝ
```

lemma potential_invariant[\[source\]](#)

```
lemma potential_invariant (φ : higgsVec) (g : gaugeGroup) :
  potential μSq (λ) (rep g φ) = potential μSq (λ) φ
```

lemma potential_as_quad[\[source\]](#)

```
lemma potential_as_quad (φ : higgsVec) :
  λ * ||φ|| ^ 2 * ||φ|| ^ 2 + (- μSq ) * ||φ|| ^ 2 + (- potential μSq (λ) φ) = 0
```

lemma potential_snd_term_nonneg[\[source\]](#)

```
lemma potential_snd_term_nonneg (φ : higgsVec) :
  0 ≤ λ * ||φ|| ^ 4
```

lemma zero_le_potential_discrim[\[source\]](#)

```
lemma zero_le_potential_discrim (φ : higgsVec) :
  0 ≤ discrim (λ) (- μSq ) (- potential μSq (λ) φ)
```

lemma potential_eq_zero_sol[\[source\]](#)

```
lemma potential_eq_zero_sol (φ : higgsVec)
  (hV : potential μSq (λ) φ = 0) : φ = 0 ∨ ||φ|| ^ 2 = μSq / λ
```

lemma potential_eq_zero_sol_of_μSq_nonpos[\[source\]](#)

```
lemma potential_eq_zero_sol_of_μSq_nonpos (hμSq : μSq ≤ 0)
  (φ : higgsVec) (hV : potential μSq (λ) φ = 0) : φ = 0
```

lemma potential_bounded_below[\[source\]](#)

```
lemma potential_bounded_below (φ : higgsVec) :
  - μSq ^ 2 / (4 * (λ)) ≤ potential μSq (λ) φ
```

lemma potential_bounded_below_of_μSq_nonpos[\[source\]](#)

```
lemma potential_bounded_below_of_μSq_nonpos {μSq : ℝ}
  (hμSq : μSq ≤ 0) (φ : higgsVec) : 0 ≤ potential μSq (λ) φ
```

lemma potential_eq_bound_discrim_zero[\[source\]](#)

```
lemma potential_eq_bound_discrim_zero (φ : higgsVec)
  (hV : potential μSq (λ) φ = - μSq ^ 2 / (4 * λ)) :
  discrim (λ) (- μSq) (- potential μSq (λ) φ) = 0
```

lemma potential_eq_bound_higgsVec_sq[\[source\]](#)

```
lemma potential_eq_bound_higgsVec_sq (φ : higgsVec)
  (hV : potential μSq (λ) φ = - μSq ^ 2 / (4 * (λ))) :
  ||φ|| ^ 2 = μSq / (2 * λ)
```

lemma potential_eq_bound_iff[\[source\]](#)

```
lemma potential_eq_bound_iff (φ : higgsVec) :
  potential μSq (λ) φ = - μSq ^ 2 / (4 * (λ)) ↔ ||φ|| ^ 2 = μSq / (2 * (λ))
```

lemma potential_eq_bound_iff_of_μSq_nonpos[\[source\]](#)

```
lemma potential_eq_bound_iff_of_μSq_nonpos {μSq : ℝ}
  (hμSq : μSq ≤ 0) (φ : higgsVec) : potential μSq (λ) φ = 0 ↔ φ = 0
```

lemma potential_eq_bound_IsMin0n[\[source\]](#)

```
lemma potential_eq_bound_IsMin0n (φ : higgsVec)
  (hV : potential μSq lambda φ = - μSq ^ 2 / (4 * lambda)) :
  IsMin0n (potential μSq lambda) Set.univ φ
```

lemma potential_eq_bound_IsMin0n_of_μSq_nonpos[\[source\]](#)

```
lemma potential_eq_bound_IsMin0n_of_μSq_nonpos {μSq : ℝ}
  (hμSq : μSq ≤ 0) (φ : higgsVec) (hV : potential μSq lambda φ = 0) :
  IsMin0n (potential μSq lambda) Set.univ φ
```

lemma potential_bound_reached_of_μSq_nonneg[\[source\]](#)

```
lemma potential_bound_reached_of_μSq_nonneg {μSq : ℝ} (hμSq : 0 ≤ μSq) :
  ∃ (φ : higgsVec), potential μSq lambda φ = - μSq ^ 2 / (4 * lambda)
```

lemma IsMinOn_potential_iff_of_μSq_nonneg[\[source\]](#)

```
lemma IsMinOn_potential_iff_of_μSq_nonneg {μSq : ℝ} (hμSq : 0 ≤ μSq) :
  IsMinOn (potential μSq lambda) Set.univ φ ↔ ||φ|| ^ 2 = μSq / (2 * lambda)
```

lemma IsMinOn_potential_iff_of_μSq_nonpos[\[source\]](#)

```
lemma IsMinOn_potential_iff_of_μSq_nonpos {μSq : ℝ} (hμSq : μSq ≤ 0) :
  IsMinOn (potential μSq lambda) Set.univ φ ↔ φ = 0
```

def rotateMatrix[\[source\]](#)

Given a Higgs vector, a rotation matrix which puts the first component of the vector to zero, and the second component to a real

```
def rotateMatrix (φ : higgsVec) : Matrix (Fin 2) (Fin 2) ℂ
```

lemma rotateMatrix_star[\[source\]](#)

```
lemma rotateMatrix_star (φ : higgsVec) :
  star φ.rotateMatrix =
  ![![conj (φ 1) / ||φ|| , φ 0 / ||φ||], ![ - conj (φ 0) / ||φ|| , φ 1 / ||φ|| ] ]
```

lemma rotateMatrix_det[\[source\]](#)

```
lemma rotateMatrix_det {φ : higgsVec} (hφ : φ ≠ 0) : (rotateMatrix φ).det = 1
```

lemma rotateMatrix_unitary[\[source\]](#)

```
lemma rotateMatrix_unitary {φ : higgsVec} (hφ : φ ≠ 0) :
  (rotateMatrix φ) ∈ unitaryGroup (Fin 2) ℂ
```

lemma rotateMatrix_specialUnitary[\[source\]](#)

```
lemma rotateMatrix_specialUnitary {φ : higgsVec} (hφ : φ ≠ 0) :
  (rotateMatrix φ) ∈ specialUnitaryGroup (Fin 2) ℂ
```

def rotateGaugeGroup[\[source\]](#)

Given a Higgs vector, an element of the gauge group which puts the first component of the vector to zero, and the second component to a real

```
def rotateGaugeGroup {φ : higgsVec} (hφ : φ ≠ 0) : gaugeGroup
```

lemma rotateGaugeGroup_apply[\[source\]](#)

```
lemma rotateGaugeGroup_apply {φ : higgsVec} (hφ : φ ≠ 0) :
  rep (rotateGaugeGroup hφ) φ = ![0, ofReal ||φ||]
```

theorem rotate_fst_zero_snd_real[\[source\]](#)

```

theorem rotate_fst_zero_snd_real (φ : higgsVec) :
  ∃ (g : gaugeGroup), rep g φ = ![0, ofReal ||φ||]

```

2.16 HepLean.StandardModel.Representations

[HepLean/StandardModel/Representations.lean](#) on GitHub.

def repU1Map[\[source\]](#)

The 2d representation of $U(1)$ with charge 3 as a map from $U(1)$ to unitaryGroup (Fin 2) $\mathbb{C}$.

```

@[simps!]
noncomputable def repU1Map (g : unitary  $\mathbb{C}$ ) : unitaryGroup (Fin 2)  $\mathbb{C}$ 

```

def repU1[\[source\]](#)

The 2d representation of $U(1)$ with charge 3 as a homomorphism from $U(1)$ to unitaryGroup (Fin 2) $\mathbb{C}$.

```

@[simps!]
noncomputable def repU1 : unitary  $\mathbb{C}$  →* unitaryGroup (Fin 2)  $\mathbb{C}$  where

```

def fundamentalSU2[\[source\]](#)

The fundamental representation of $SU(2)$ as a homomorphism to unitaryGroup (Fin 2) $\mathbb{C}$.

```

@[simps!]
def fundamentalSU2 : specialUnitaryGroup (Fin 2)  $\mathbb{C}$  →* unitaryGroup (Fin 2)  $\mathbb{C}$  where

```

lemma repU1_fundamentalSU2_commute[\[source\]](#)

```

lemma repU1_fundamentalSU2_commute (u1 : unitary  $\mathbb{C}$ ) (g : specialUnitaryGroup (Fin 2)  $\mathbb{C}$ ) :
  repU1 u1 * fundamentalSU2 g = fundamentalSU2 g * repU1 u1

```